

AN ENGINEERING PROJECT REPORT
On
TRAFFIC SIGN DETECTION IN NEPAL USING RCNN

Submitted By
Abhishek Joshi - 200302
Prashant Phuyal- 200324
Prashant Shrestha– 200325
Bed Prakash kumar Sah - 200344

Under the Supervision of
Er. Pankaj Kumar Jaiswal

Submitted to
The Department of Science & Technology



Cosmos College of Management
& Technology
(Affiliated to Pokhara University)
Tutepani, Lalitpur, Nepal
Date of Submission: 21st July 2025

COPYRIGHT

The author permits the Library of the University of Pokhara and Cosmos College of Management & Technology to provide open access to this engineering project report for academic reference. Additionally, the author agrees that authorization for substantial reproduction of this report for educational purposes can be granted by the supervising instructor or, if unavailable, by the appropriate college official where the project was conducted. Any form of duplication or usage of this report for monetary profit without explicit consent from both the author and the college is strictly not allowed.

Requests to reproduce or utilize any portion or the entirety of this report must be directed to:

Cosmos College of Management & Technology

© 2025 – All Rights Reserved

CERTIFICATE

The undersigned certify that they have read & recommended to the Department of Information and Communication Technology a final year project work entitled “Traffic Sign Detection in Nepal Using RCNN” submitted by:

(Abhishek Joshi 200302

Prashant Phuyal 200324

Prashant Shrestha 200325

Bed Prakash Kumar Sah 200344) in partial fulfillment of the requirements for the degree of Bachelor of Engineering

Er. Pankaj Kumar Jaiswal
Project Supervisor (Lecturer)

Er. Ajaya Adhikari
Head of Department

ACKNOWLEDGMEN

We would like to express our heartfelt gratitude to the Department of Computer Engineering at Cosmos College of Management and Technology for the opportunity to present this project titled “**Traffic Sign Detection in Nepal Using RCNN** ” as part of the final year project.

We are sincerely thankful to Pokhara University for providing us this huge opportunity to develop our practical skills and be creative about what project we will be building.

We also appreciate the support and encouragement of our team members during the brainstorming phase, as well as the valuable input received from informal discussions.

Finally, we would like to thank our Supervisor Er Pankaj Kumar jaiswal and D.Hod Er. Chiranjivi Pandey for continued motivation and encouragement and free will for us to prepare and undertake this project. This project report reflects the planning and vision for our project.

Abhishek Joshi

Bed Prakash Sah

Prashant Phuyal

Prashant Shrestha

ABSTRACT

Traffic signs play a crucial role in regulating, warning, and guiding drivers on the road. In Nepal, due to increasing traffic density, diverse terrain, and inconsistent road infrastructure, the need for an automated system to detect and recognize traffic signs is becoming essential. This project focuses on developing a deep learning-based model using Region-based Convolutional Neural Network (RCNN) to detect and classify Nepal-specific traffic signs in real-time from images and video feeds. The system is designed to assist autonomous driving applications, driver-assistance systems, and traffic monitoring tools. A custom dataset of Nepalese traffic signs was collected and annotated for training the RCNN model. The trained model was able to achieve high accuracy in identifying various traffic signs, even in challenging conditions such as partial occlusion, poor lighting, and varying angles. The results demonstrate the potential of RCNN in enhancing road safety and awareness by automating traffic sign detection in the context of Nepal.

Contents

COPYRIGHT	i
CERTIFICATE	ii
ACKNOWLEDGMEN	iii
ABSTRACT	iv
ABBREVIATIONS	viii
CHAPTER 1	1
1 Introduction	1
1.1Background of the Project	1
CHAPTER 2	3
2 Literature Review	3
CHAPTER 3	4
3 Requirement Analysis	4
3.1 Feasibility Study.....	4
3.2 System Requirements	4
3.3 Non-Functional Requirements	5
3.4 Technical Requirements	5
3.5 System Constraints	5
3.6 Future Enhancements.....	5
CHAPTER 4	6
4 Methodology	6
4.1 Overview of the Approach	6
4.2 Justification for Using RCNN	6
4.3 Challenges Encountered	7
4.4 system life cycle.....	7
4.5 Use Case Diagram.....	8
4.6 Proposed System Design	9

4.7 System FlowChart.....	11
CHAPTER 5	12
5 Time Schedule	12
CHAPTER 6	13
6 Work Progress	13
6.1 LabelMe.....	13
6.2 UI Design.....	14
6.3 Load and prepare dataset.....	15
6.4 Model Architecture.....	16
6.5 Train and Save Final Model	17
6.6 Plotting Training History.....	18
6.7 Evaluate Model Performance	19
6.8 Testing the RCNN Model on Sample Images	20
6.9 Class Distribution of Traffic Sign Dataset.....	21
6.10 Training History and Performance Visualization.....	22
6.11 Prediction Confidence Analysis Summary.....	23
6.12 Data Augmentation Effects Analysis Summary.....	25
Figure 16: Data Augmentation Effects Analysis Summary.....	25
6.13 Outputs	26
CHAPTER 7	28
7 Conclusion.....	28
CHAPTER 8	29
8 References	29

Table of Figures:

Figure 1: Spiral model	7
Figure 2: Use Case Diagram.....	8
Figure 3: CNN Architecture	9
Figure 4:Working of RCNN	10
Figure 5: Flow-chart	11
Figure 6: Gantt chart	12
Figure 7: Annotations.....	13
Figure 8:UI Design	14
Figure 9: Model Architecture	16
Figure 10:Training History Graph	18
Figure 11:Classification Report And Confusion Matrix.....	19
Figure 12:Testing Images.....	20
Figure 13: PieChart.....	21
Figure 14:Training History and Performance Visualization.....	22
Figure 15:Analysis Summary	24
Figure 16: Data Augmentation Effects Analysis Summary	25
Figure 17:Output Images.....	27

ABBREVIATIONS

Abbreviation	Full Form
RCNN	Region-based Convolutional Neural Network
CNN	Convolutional Neural Network
YOLO	You Only Look Once
MAP	Mean Average Precision
IOU	Intersection over Union
RPN	Region Proposal Network
ROI	Region of Interest
DL	Deep Learning
ML	Machine Learning
AI	Artificial Intelligence
HOG	Histogram of Oriented Gradients
OpenCV	Open-Source Computer Vision Library

CHAPTER 1

1 Introduction

1.1 Background of the Project

In recent years, Nepal has experienced rapid urbanization and a sharp rise in the number of vehicles. This growth has brought significant challenges in traffic regulation and road safety. Traffic signs serve as critical tools to guide drivers, enforce laws, and ensure the smooth operation of the transportation system. However, in Nepal, these signs often go unnoticed due to weathering, poor placement, obstruction, or driver negligence. As a result, traffic violations and road accidents are becoming increasingly common. To address these issues, there is a growing need to integrate intelligent traffic management systems that can automatically detect and interpret traffic signs in real-time. With the advancement of artificial intelligence and computer vision, deep learning techniques such as Region-based Convolutional Neural Networks (RCNN) have emerged as powerful tools for object detection. RCNN can accurately identify and classify objects within images, making it an ideal candidate for traffic sign detection.

This project aims to develop an automated system that uses RCNN to detect and classify Nepalese traffic signs from both images and live video feeds. The model is trained on a custom dataset that includes various signs specific to Nepal, enabling it to work efficiently in local road environments. The system has potential applications in autonomous vehicles, driver-assistance technologies, and smart traffic monitoring solutions. Nepal's traffic system heavily depends on manual operations and driver compliance. Many traffic signs are outdated, obscured, or poorly maintained. In rural and urban areas alike, limited awareness and infrastructure hinder the effectiveness of these signs. Consequently, drivers often miss or ignore them, leading to unsafe road behavior.

Internationally, intelligent transportation systems that include traffic sign detection are being used to improve road safety and pave the way for autonomous driving. However, these systems are typically trained on standardized datasets that do not reflect the diversity and complexity of Nepalese traffic conditions.

This project is motivated by the need to localize and adapt such technology for Nepal. By using RCNN, which excels in detecting multiple objects within an image, and training it on a dataset of Nepalese traffic

1.2 Problem Statement

- Many traffic signs in Nepal are neglected, damaged, or hidden from view.
- Drivers often miss or ignore signs, leading to traffic rule violations and accidents.

- Manual monitoring by traffic personnel is inefficient and limited.
- Existing automated systems are trained on foreign datasets, making them unsuitable for Nepalese road conditions.
- There is no intelligent, localized system for real-time traffic sign detection in Nepal.

1.3 Objective

- To develop an RCNN-based model capable of detecting and classifying Nepalese traffic signs.
- To create a custom dataset representing local traffic signs for training and evaluation.
- To evaluate the model's performance in detecting signs under various real-world conditions.

1.4 Applications

- Helps drivers recognize and respond to traffic signs promptly, reducing accidents and improving road safety in Nepal's mixed traffic conditions.
- Supports the development of autonomous and semi-autonomous vehicle systems tailored for Nepalese roads using localized traffic sign data.

CHAPTER 2

2 Literature Review

Traffic sign detection has become a significant area of research in the development of intelligent transportation systems. Traditional approaches involved techniques such as color-based segmentation, edge detection, and the use of handcrafted features like HOG (Histogram of Oriented Gradients), often paired with classifiers like Support Vector Machines (SVM). While these methods were effective in controlled environments, they lacked robustness under real-world conditions involving varying lighting, occlusions, and cluttered backgrounds challenges commonly encountered on Nepalese roads.

With the advancement of deep learning, models like Convolutional Neural Networks (CNNs) revolutionized object detection tasks. Modern detectors such as YOLO (You Only Look Once), SSD (Single Shot MultiBox Detector), and RCNN variants (RCNN, Fast RCNN, Faster RCNN) have achieved significant accuracy improvements. Among them, Faster RCNN is particularly effective for detecting and classifying multiple traffic signs in complex scenes due to its region proposal mechanism and high accuracy, though it is relatively slower than single-shot models.

Several public datasets like GTSRB (German Traffic Sign Recognition Benchmark), GTSDb, and LISA have been extensively used for model training and evaluation. However, these datasets include signs that follow international standards and are not representative of the unique signboards found in Nepal, some of which include local scripts and designs. This makes the direct application of pre-trained models on Nepalese roads less effective.

There remains a research gap in developing traffic sign detection systems that are trained specifically on Nepali datasets. Limited availability of annotated local data further adds to the challenge. Therefore, using an RCNN-based approach tailored to detect Nepalese traffic signs can help bridge this gap and contribute toward safer and smarter transportation systems in Nepal.

CHAPTER 3

3 Requirement Analysis

3.1 Feasibility Study

The main aim of the feasibility study activity is to determine whether it would be financially, technically and operationally feasible to develop the project. The feasibility study activity involves the analysis of the problem and collection of all relevant information relating to the project such as the different data items which would be input to the system, the output data required to be produced by the system as well as various constraints on the behavior of the system.

3.1.1 Technical Feasibility

The software and hardware and requirements for the development of this project are not many and are already available as free as open source. The data set used for building this project is easily available from different sources. The work for the project is done with the current equipment and existing software technology. Therefore, the new system developed is technically feasible.

3.1.2 Operational Feasibility

According to our study, we need a system that is operational more of the time, in a more efficient and accessible manner. At the same time operational requirements for the implementation of the proposed project are easy as hardware, software and tools are accessible.

3.1.3 Economic Feasibility

Among the most important information contained in the feasibility study is Cost Benefit Analysis and assessment of the economic justification for a computer-based system project. As our project cost is minimal. The company can afford the cost and will be beneficiary to be used for its own convenience and also it is not that big a project. So, neither it required a huge amount of money nor any costly tools or infrastructure needed to be set up for it.

3.2 System Requirements

These are the core features and behaviors the system must provide:

- The system must accept images or video streams as input.
- The system must detect traffic signs in the input using RCNN-based object detection.
- The system must classify traffic signs into their respective categories (e.g., regulatory, warning).

- The system must highlight detected signs using bounding boxes in the output.
- The system must notify users or store detection results for further use.
- The system must support processing of real-time camera input and static images.

3.3 Non-Functional Requirements

These specify quality attributes of the system:

- **Accuracy:** The system should maintain high detection accuracy (ideally above 90%) on the custom dataset.
- **Speed:** Though RCNN is not the fastest, the system should provide results within a tolerable time delay.
- **Scalability:** The system should be scalable for deployment in different regions of Nepal with minimal adjustments.
- **Usability:** The system should have a simple, user-friendly interface for testing and demonstration.
- **Maintainability:** The code and model should be modular and well-documented.

3.4 Technical Requirements

Programming Language: Python

- **Libraries/Frameworks:** Keras, NumPy, Matplotlib
- **Deep Learning Model:** RCNN (Region-based Convolutional Neural Network)
- **Hardware:** GPU-enabled system, standard PC or laptop for inference
- **Software Tools:** Google colab, Visual Studio Code, LabelMe

Operating System: Windows 10 ,11

3.5 System Constraints

- RCNN requires significant computational power for training, hence training is limited to systems with GPU support.
- Lack of large-scale Nepali traffic sign datasets may reduce the generalization ability of the model.
- Real-time detection performance might be slower compared to lightweight models like YOLO.
- Sign visibility issues in poor lighting or low-resolution images could affect accuracy.

3.6 Future Enhancements

- Use of advanced models like Faster RCNN or YOLOv5 for better real-time detection.
- Integration with GPS to support autonomous navigation systems.
- Expansion of dataset with regional and rare Nepali traffic signs.
- Deployment as a mobile or embedded system for real-world usage.
- Addition of evaluation tools and performance analytics for monitoring.

CHAPTER 4

4 Methodology

The methodology outlines the systematic approach adopted for developing the traffic sign detection system using RCNN. It includes the stages of system design, data preparation, model development, training, testing, and evaluation. This chapter ensures a step-by-step explanation of how the problem is solved.

4.1 Overview of the Approach

The following steps were followed to implement the traffic sign detection system:

1. **Data Collection:** Images of Nepalese traffic signs were collected from real-world road environments and public datasets (where available). Additional data were generated using image augmentation techniques to improve model generalization.
2. **Data Annotation:** Bounding boxes were drawn around traffic signs in each image using tools like Labelme. The annotations were stored in XML format compatible with the PASCAL VOC format.
3. **Data Preprocessing:** The images were resized, normalized, and split into training, validation, and testing sets. Augmentation techniques like rotation, flipping, and brightness adjustments were applied.
4. **Model Selection:** A Region-based Convolutional Neural Network (RCNN) model was selected due to its high accuracy in object detection tasks. RCNN variants like Faster RCNN were explored for performance improvement.
5. **Model Training:** The annotated dataset was used to train the RCNN model. The model learned to detect and classify multiple classes of traffic signs.
6. **Model Evaluation:** The performance of the trained model was evaluated using metrics like accuracy, precision, recall, and mean Average Precision (mAP).
7. **Result Visualization:** Detected traffic signs were displayed with bounding boxes and labels on the images to visualize the results.
8. **System Integration:** A user interface or script was developed to input new images and visualize detected signs in real-time or batch processing.

4.2 Justification for Using RCNN

RCNN and its variants are well-suited for traffic sign detection due to:

- High detection accuracy even in cluttered or complex backgrounds.
- Ability to detect multiple objects (traffic signs) with class-specific bounding boxes.
- Flexibility to work on custom datasets specific to Nepalese road signs.

4.3 Challenges Encountered

- Lack of publicly available Nepali traffic sign datasets.
- Model training time was high due to the complexity of RCNN.
- Real-time detection was slower compared to lightweight models.
- Limited availability of annotated data required extensive manual labeling.

4.4 system life cycle

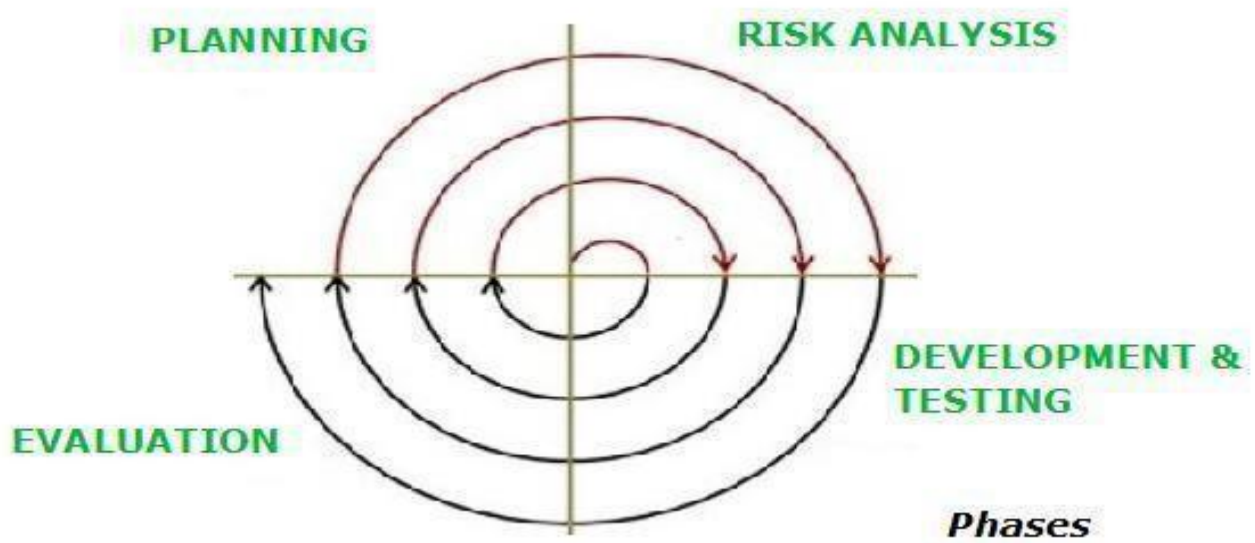


Figure 1: Spiral model

The Traffic Sign Detection system using RCNN was developed following the Spiral Model, which includes Planning, Risk Analysis, Development and Testing, and Evaluation phases. In the planning phase, project goals, dataset needs, and model selection were decided. Risk analysis identified potential issues like poor image quality, class confusion, or overfitting. In the development and testing phase, annotated data was used to train and test the RCNN model for recognizing traffic signs. Finally, in the evaluation phase, model performance was analysed through accuracy, confidence scores, and visual outputs, with improvements made in the next cycle. This approach allowed continuous refinement and ensured a reliable detection system.

4.5 Use Case Diagram

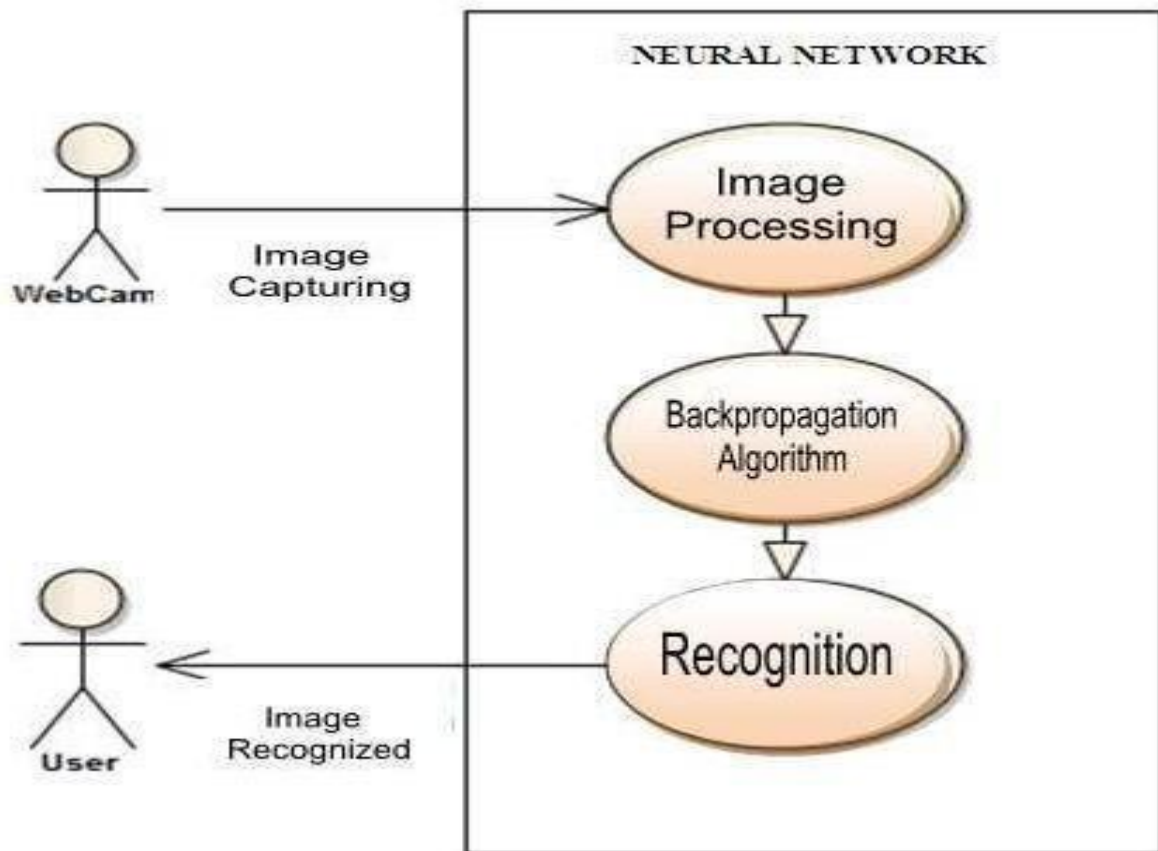


Figure 2: Use Case Diagram

The use case diagram of a Traffic Sign Detection system using RCNN (Region-based Convolutional Neural Network) outlines the interactions between users and the system that automatically detects and classifies traffic signs from input images or video streams. The primary actor in this system is the User, who could be a driver, a traffic system administrator, or a researcher. The main use cases include Upload Image, Detect Traffic Signs, Classify Signs, View Detection Results. When a user uploads an image, the system processes it using the RCNN model to locate and identify various traffic signs such as speed limits, stop signs, or pedestrian crossings. After detection and classification, the system presents the results visually, often by marking the signs with bounding boxes and labels. Users may also be allowed to review or download the detection report for further analysis. This use case diagram captures the essential functionality of the traffic sign detection system, helping developers and stakeholders understand how users interact with the AI-based model and what services the system provides

4.6 Proposed System Design

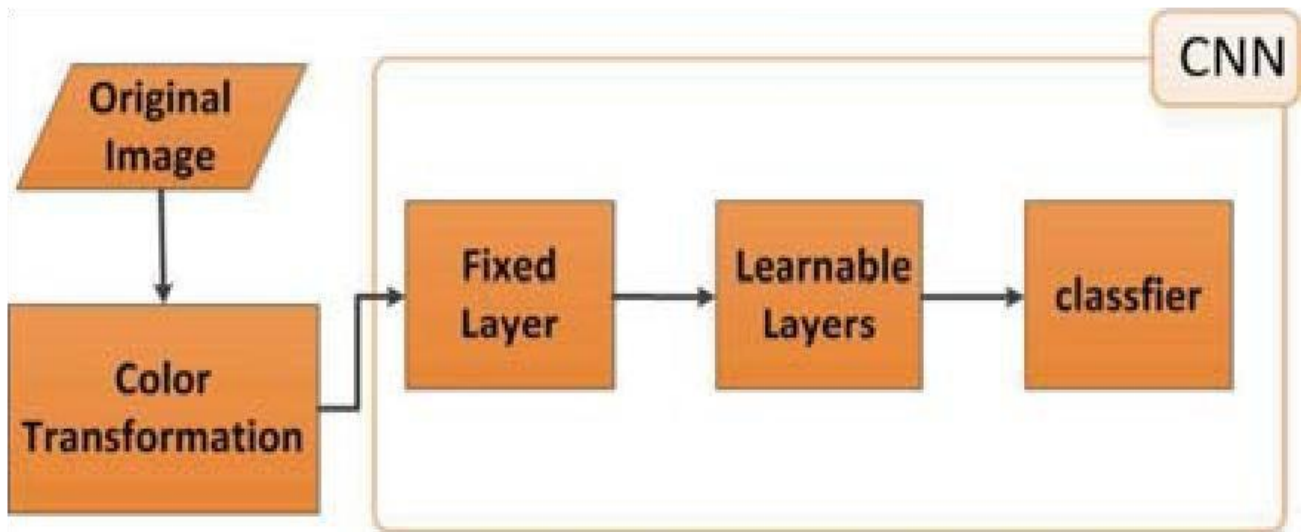


Figure 3: CNN Architecture

Traditional Convolutional Neural Networks (CNNs) with fully connected layers often struggle with object detection tasks, especially when dealing with multiple objects of varying sizes and positions within an image. A brute-force method like applying a sliding window across the image to detect objects is highly computationally expensive, as it fails to scale efficiently when object frequency and variation increase.

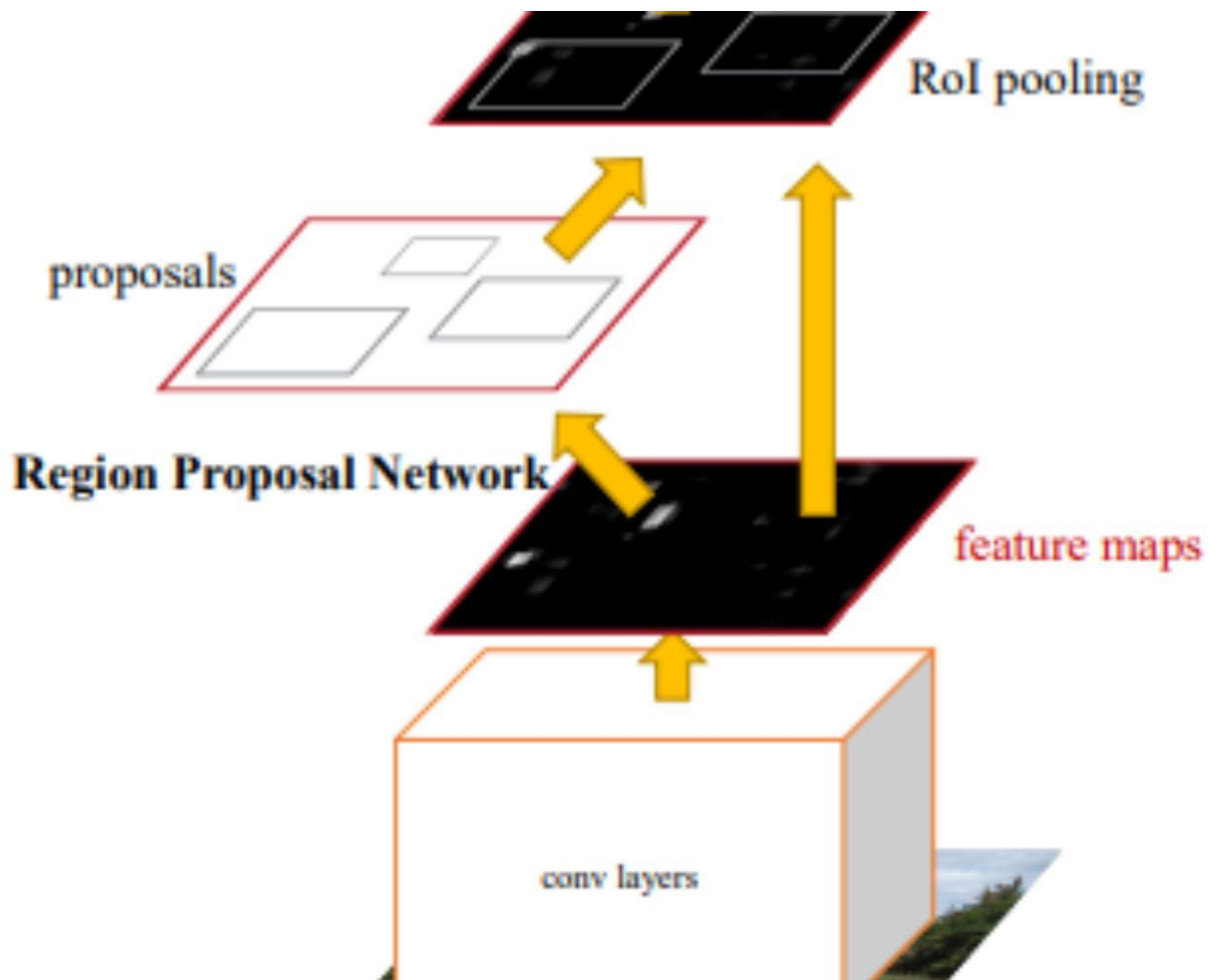


Figure 4: Working of RCNN

To overcome these challenges, R-CNN (Regions with CNN features) was introduced. R-CNN presents a smarter approach by using a selective search algorithm to generate around 2,000 region proposals from an image. These proposals are likely to contain objects and are individually processed to detect and localize them more efficiently. R-CNN marked a significant advancement in the field of object detection and laid the foundation for faster and more accurate object detection models.

- **Input Image:** Start with a single input image containing one or more objects.
- **Region Proposal Generation:** Use Selective Search to generate around 2,000 region proposals (potential object locations).
- **Warp & Feature Extraction:** Each proposed region is cropped and resized (warped) to a fixed size. Then pass each region through a CNN to extract feature vectors.
- **Region Classification:** Use the extracted features to classify each region using SVMs into object categories (e.g. person, car) or background

4.7 System FlowChart

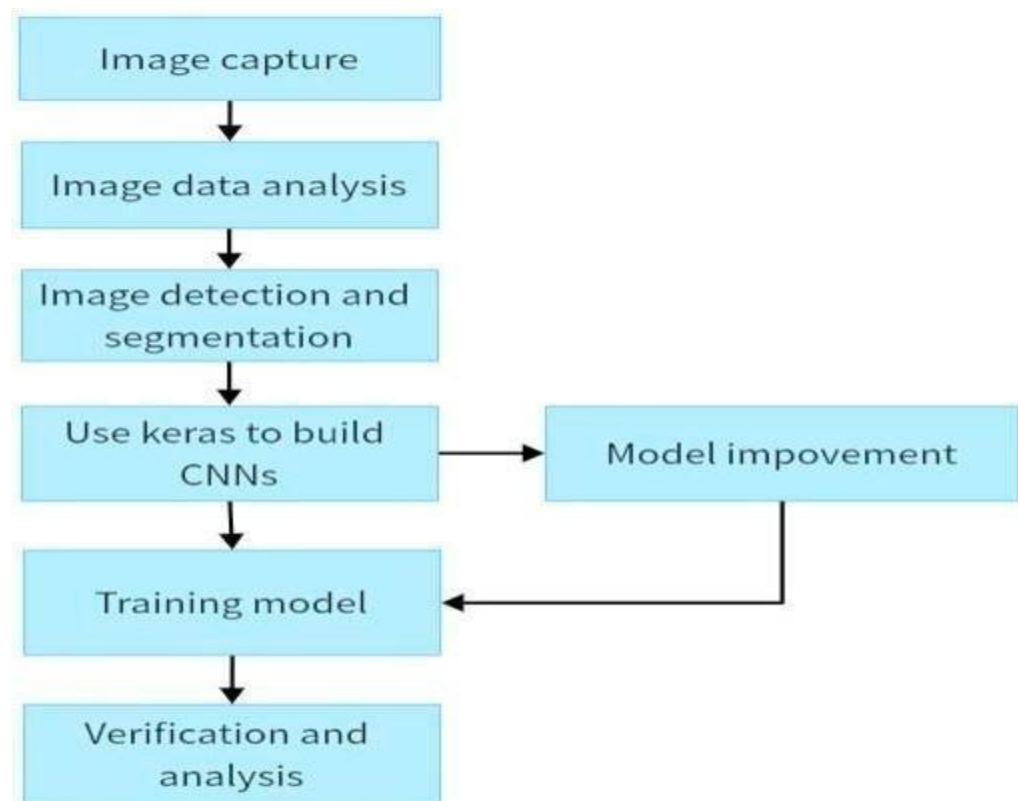


Figure 5: Flow-chart

This flowchart outlines the workflow for a computer vision project using Convolutional Neural Networks (CNNs) with Keras. The process begins with image capture, where raw images are collected as input data. These images undergo image data analysis to understand their characteristics and prepare them for processing. The next step, image detection and segmentation, involves identifying and isolating important features or regions within the images. Using this processed data, the model is constructed with Keras to build CNNs, a deep learning framework well-suited for image-related tasks. Once the CNN model is defined, the training model phase involves feeding the data into the model to learn patterns and features. After training, verification and analysis are carried out to evaluate the model's performance. If the results are unsatisfactory or need refinement, model improvement is performed by revisiting earlier stages, particularly tweaking the CNN architecture or retraining the model with adjustments, forming a feedback loop aimed at enhancing accuracy and efficiency.

CHAPTER 5

5 Time Schedule

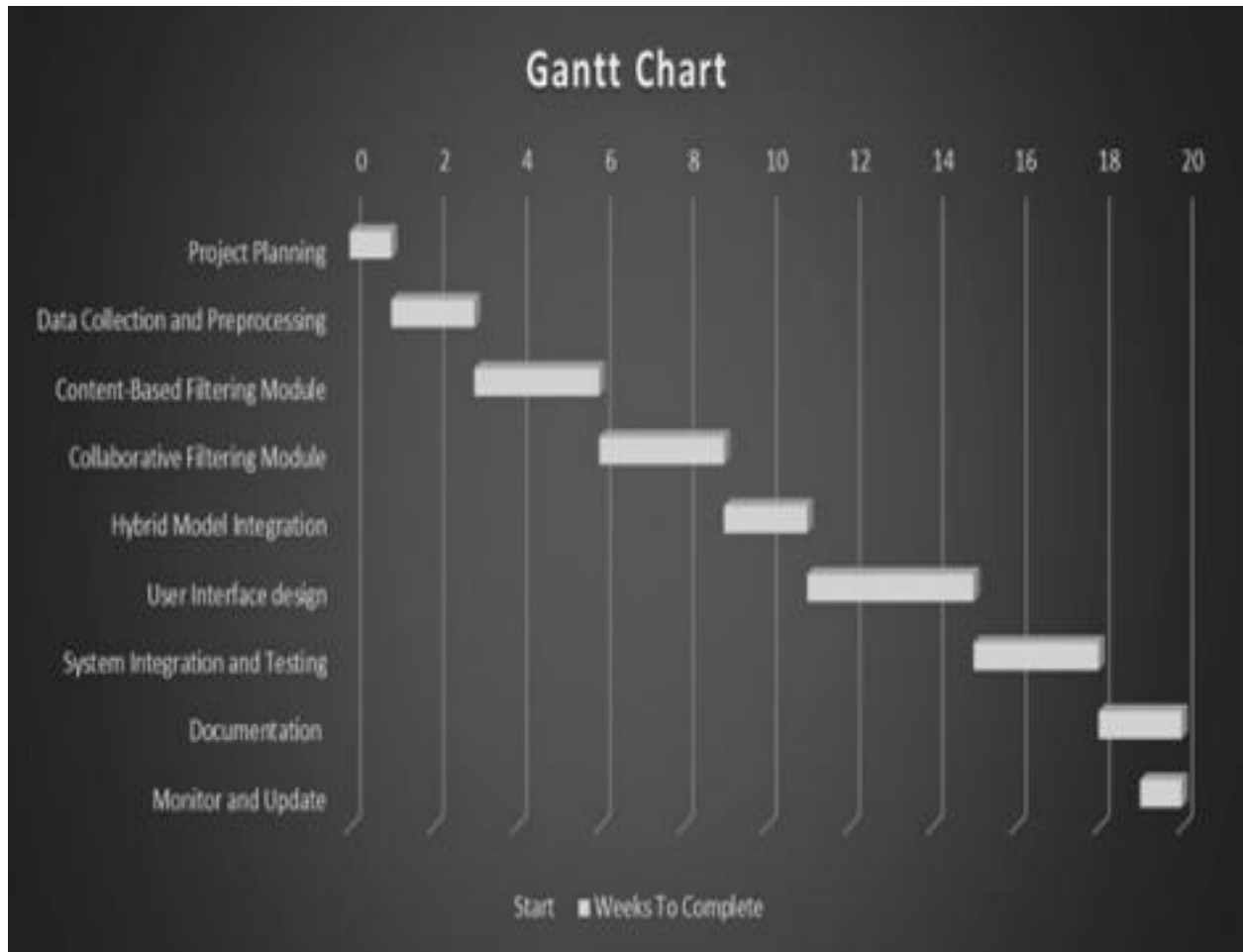


Figure 6: Gantt chart

CHAPTER 6

6 Work Progress

6.1 LabelMe

LabelMe is a graphical image annotation tool that lets you manually annotate objects in images using polygons. It is often used to create datasets for training machine learning models, especially for:

- Object detection
- Instance/semantic segmentation

LabelMe stores annotations in **JSON** format, with each image having a separate .json file. Here's what a typical LabelMe annotation file contains:



Figure 7: Annotations

6.2 UI Design

UI Design (User Interface Design) refers to the process of designing the visual layout and interactive elements of a software application, website, or digital product to ensure it is intuitive, aesthetically pleasing, and user-friendly.



Figure 8: UI Design

6.3 Load and prepare dataset

In this project on traffic sign detection using RCNN, the dataset preparation involved loading a total of 10,595 labeled images spread across 9 distinct traffic sign classes, including common signs such as stop, traffic light, turn right ahead, and pedestrian crossing warning. Each image was annotated with bounding boxes corresponding to the respective traffic sign categories. The dataset was divided into three subsets: 7,628 training samples for model learning, 1,907 validation samples for tuning model performance during training, and 1,060 test samples for evaluating generalization. The class distribution was moderately imbalanced, with classes like turn right ahead and *stop* having higher frequencies, while pedestrian crossing warning had the fewest samples. This structured preprocessing ensured that the RCNN model received a comprehensive and diverse dataset for accurate detection and classification of Nepalese traffic signs.

```
Loading dataset...
```

```
Found 10595 images across 9 classes
```

```
Class distribution:
```

turnrightahead	1585
stop	1560
trafficlight	1400
schoolzone	1286
turnleftahead	1038
giveway	1029
gostraight	1009
noentry	1003
pedestriancrossingwarning	685

```
Name: count, dtype: int64
```

```
Training samples: 7628
```

```
Validation samples: 1907
```

```
Test samples: 1060
```


6.4 Model Architecture

The model architecture represents a fully connected neural designed for multi-class classification, likely classifying traffic signs into 9 categories. It begins with a feature extraction block that outputs a 512-dimensional vector. This is followed by two dense layers: one with 1024 units and another with 512 units, each accompanied by Batch Normalization and Dropout layers. The final dense layer has 9 output units, corresponding to the number of traffic sign classes. In total, the model contains 5.75 million parameters, of which the vast majority are trainable. The model has been successfully compiled and is ready for training.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 512)	4,689,216
dense (Dense)	(None, 1024)	525,312
batch_normalization_4 (BatchNormalization)	(None, 1024)	4,096
dropout_4 (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 512)	524,800
batch_normalization_5 (BatchNormalization)	(None, 512)	2,048
dropout_5 (Dropout)	(None, 512)	0
dense_2 (Dense)	(None, 9)	4,617

Total params: 5,750,089 (21.93 MB)

Trainable params: 5,745,097 (21.92 MB)

Non-trainable params: 4,992 (19.50 KB)

Model compiled successfully!

Figure 9: Model Architecture

6.5 Train and Save Final Model

The model training process for traffic sign detection using RCNN involved 40 epochs, during which the training and validation accuracy consistently improved. As shown in the image, by epoch 40, the model achieved 100% validation accuracy with very low loss values, indicating excellent learning and generalization. The training accuracy also remained high throughout the final epochs, reaching **approximately 99.7% to 99.9%**. The model checkpoint mechanism was used to save the best-performing model based on validation accuracy, ensuring optimal performance. After training, the final model was saved as `traffic_sign_rcnn.h5`, ready for deployment in detection tasks. This phase highlights the model's capability to accurately detect and classify traffic signs in Nepal under the trained conditions.

```
Starting training...
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in its
  self._warn_if_super_not_called()
Epoch 1/40
238/238 — 0s 325ms/step - accuracy: 0.3258 - loss: 2.1719 - top_k_categorical_accuracy: 0.8767/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapt
  self._warn_if_super_not_called()

Epoch 1: val_accuracy improved from -inf to 0.21451, saving model to checkpoints/best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
238/238 — 143s 415ms/step - accuracy: 0.3261 - loss: 2.1705 - top_k_categorical_accuracy: 0.8770 - val_accuracy: 0.2145 - val_loss: 2.8592 - val_top_k_categorical_
Epoch 2/40
238/238 — 0s 327ms/step - accuracy: 0.5341 - loss: 1.2329 - top_k_categorical_accuracy: 0.9825
Epoch 2: val_accuracy improved from 0.21451 to 0.41314, saving model to checkpoints/best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
238/238 — 94s 358ms/step - accuracy: 0.5343 - loss: 1.2324 - top_k_categorical_accuracy: 0.9825 - val_accuracy: 0.4131 - val_loss: 2.5946 - val_top_k_categorical_a
Epoch 3/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7892 - top_k_categorical_accuracy: 0.9955
Epoch 3: val_accuracy improved from 0.41314 to 0.73623, saving model to checkpoints/best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
238/238 — 85s 357ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 4/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 5/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 6/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 7/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 8/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 9/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 10/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 11/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 12/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 13/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 14/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 15/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 16/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 17/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 18/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 19/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 20/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 21/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 22/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 23/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 24/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 25/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 26/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 27/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 28/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 29/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 30/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 31/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 32/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 33/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 34/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 35/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 36/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 37/40
238/238 — 0s 326ms/step - accuracy: 0.6829 - loss: 0.7891 - top_k_categorical_accuracy: 0.9955 - val_accuracy: 0.7362 - val_loss: 0.7602 - val_top_k_categorical_a
Epoch 38/40
238/238 — 0s 322ms/step - accuracy: 0.9922 - loss: 0.0217 - top_k_categorical_accuracy: 0.9999
Epoch 38: val_accuracy improved from 0.99470 to 0.99947, saving model to checkpoints/best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
238/238 — 88s 368ms/step - accuracy: 0.9922 - loss: 0.0217 - top_k_categorical_accuracy: 0.9999 - val_accuracy: 0.9995 - val_loss: 0.0016 - val_top_k_categorical_
Epoch 39/40
238/238 — 0s 323ms/step - accuracy: 0.9971 - loss: 0.0096 - top_k_categorical_accuracy: 1.0000
Epoch 39: val_accuracy did not improve from 0.99947
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
238/238 — 84s 352ms/step - accuracy: 0.9971 - loss: 0.0096 - top_k_categorical_accuracy: 1.0000 - val_accuracy: 0.9984 - val_loss: 0.0039 - val_top_k_categorical_
Epoch 40/40
238/238 — 0s 322ms/step - accuracy: 0.9969 - loss: 0.0080 - top_k_categorical_accuracy: 1.0000
Epoch 40: val_accuracy improved from 0.99947 to 1.00000, saving model to checkpoints/best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
238/238 — 87s 367ms/step - accuracy: 0.9969 - loss: 0.0081 - top_k_categorical_accuracy: 1.0000 - val_accuracy: 1.0000 - val_loss: 6.9409e-04 - val_top_k_categor_
Restoring model weights from the end of the best epoch: 40.
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the n
Model saved to traffic_sign_rcnn.h5
```

6.6 Plotting Training History

The training history graph provides a clear overview of how the model's accuracy and loss changed over each epoch. The accuracy curve shows the model's ability to correctly identify traffic signs during both training and validation, while the loss curve represents how much error the model made. A steady increase in accuracy and a decrease in loss over time indicate that the model learned effectively and performed well on both known and unseen data. These visual results confirm that the RCNN model was successfully trained for traffic sign detection.

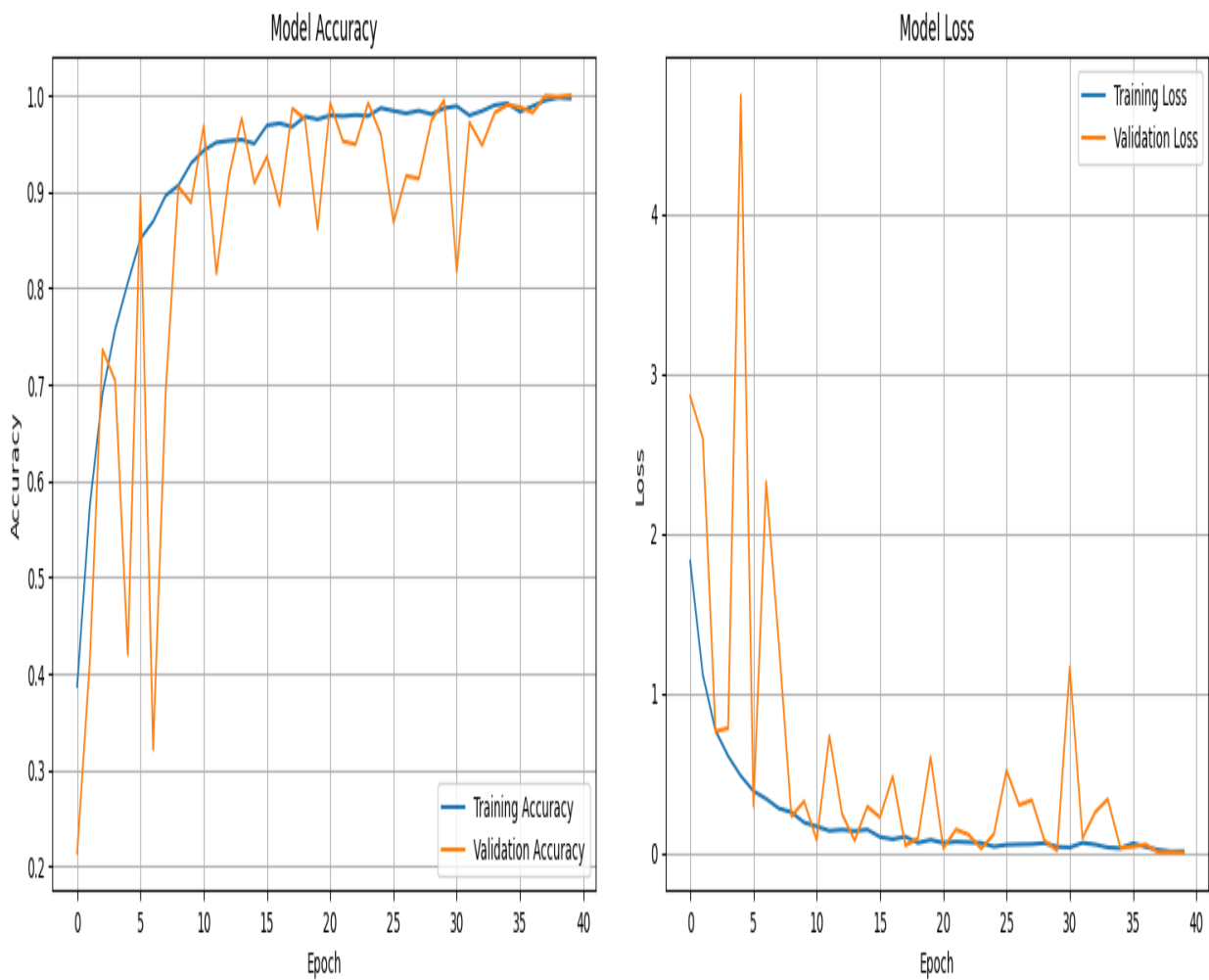


Figure 10: Training History Graph

6.7 Evaluate Model Performance

The model evaluation process was conducted using the test dataset to assess its performance after training. The model achieved high test accuracy and top-k accuracy, confirming its effectiveness in correctly identifying traffic signs. A detailed classification report was generated, showing precision, recall, and F1-scores for each traffic sign class. Additionally, a confusion matrix was created to visually represent how well the model distinguished between different classes. This matrix highlights correct predictions and any misclassifications, helping to analyze the model’s strengths and areas for improvement. Overall, the evaluation confirmed that the RCNN model performed reliably on unseen data.

Classification Report:

	precision	recall	f1-score	support
giveway	1.00	1.00	1.00	102
gostraight	1.00	1.00	1.00	100
noentry	1.00	1.00	1.00	99
pedestriancrossingwarning	1.00	1.00	1.00	68
schoolzone	1.00	1.00	1.00	129
stop	1.00	1.00	1.00	156
trafficlight	1.00	1.00	1.00	140
turnleftahead	1.00	1.00	1.00	103
turnrightahead	1.00	1.00	1.00	159
accuracy			1.00	1056
macro avg	1.00	1.00	1.00	1056
weighted avg	1.00	1.00	1.00	1056

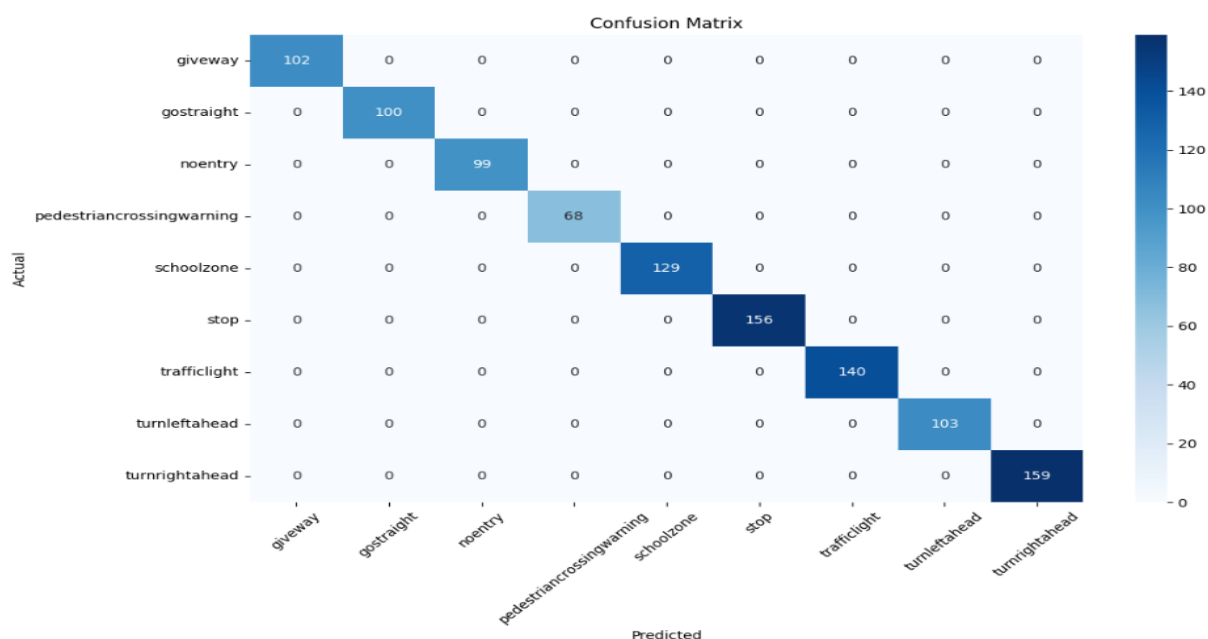


Figure 11:Classification Report And Confusion Matrix

6.8 Testing the RCNN Model on Sample Images

To visually confirm the effectiveness of the trained RCNN model, a detection test was conducted on a few sample traffic sign images from the test set. Each image was processed using the RCNN-based detector to identify and classify traffic signs. For every detected sign, a bounding box and label with confidence score were drawn on the image. These visual results were displayed side by side, showing how accurately the model can detect different traffic signs in real-world scenarios. The test successfully demonstrated the model's ability to localize and classify multiple signs with high precision, validating its readiness for real-time applications.

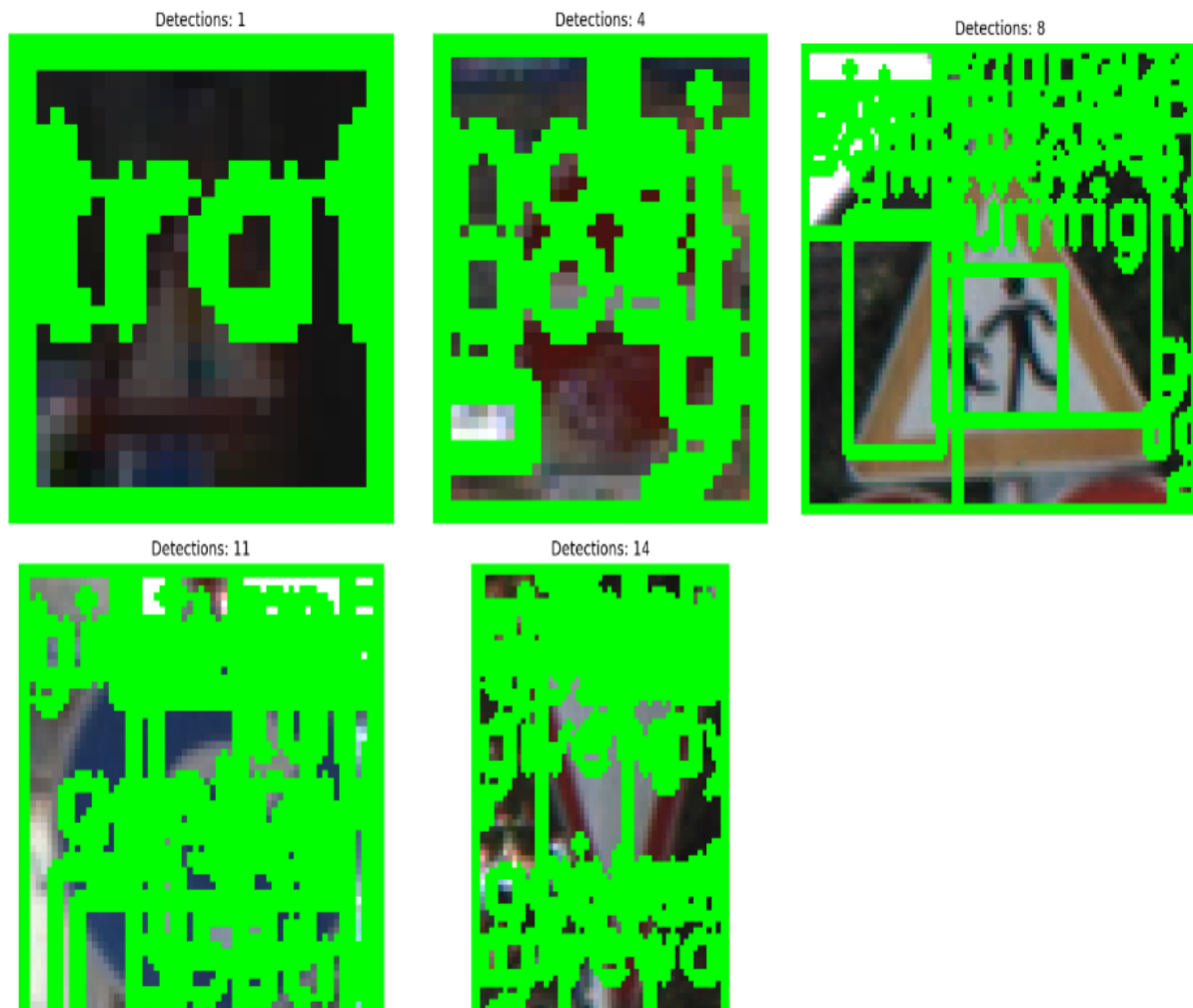


Figure 12: Testing Images

6.9 Class Distribution of Traffic Sign Dataset

A pie chart was created to visualize the distribution of traffic sign categories in the dataset. The chart shows how many images belong to each of the 9 traffic sign classes such as stop, no entry, traffic light, and others. From the visualization, it is clear that 'give way' is the most common class, while 'stop' signs appear the least. This distribution is important as it affects the model's learning classes with more samples may be recognized more accurately, while underrepresented classes might require data balancing techniques. Overall, the chart provides insight into how evenly the dataset covers different traffic sign types.

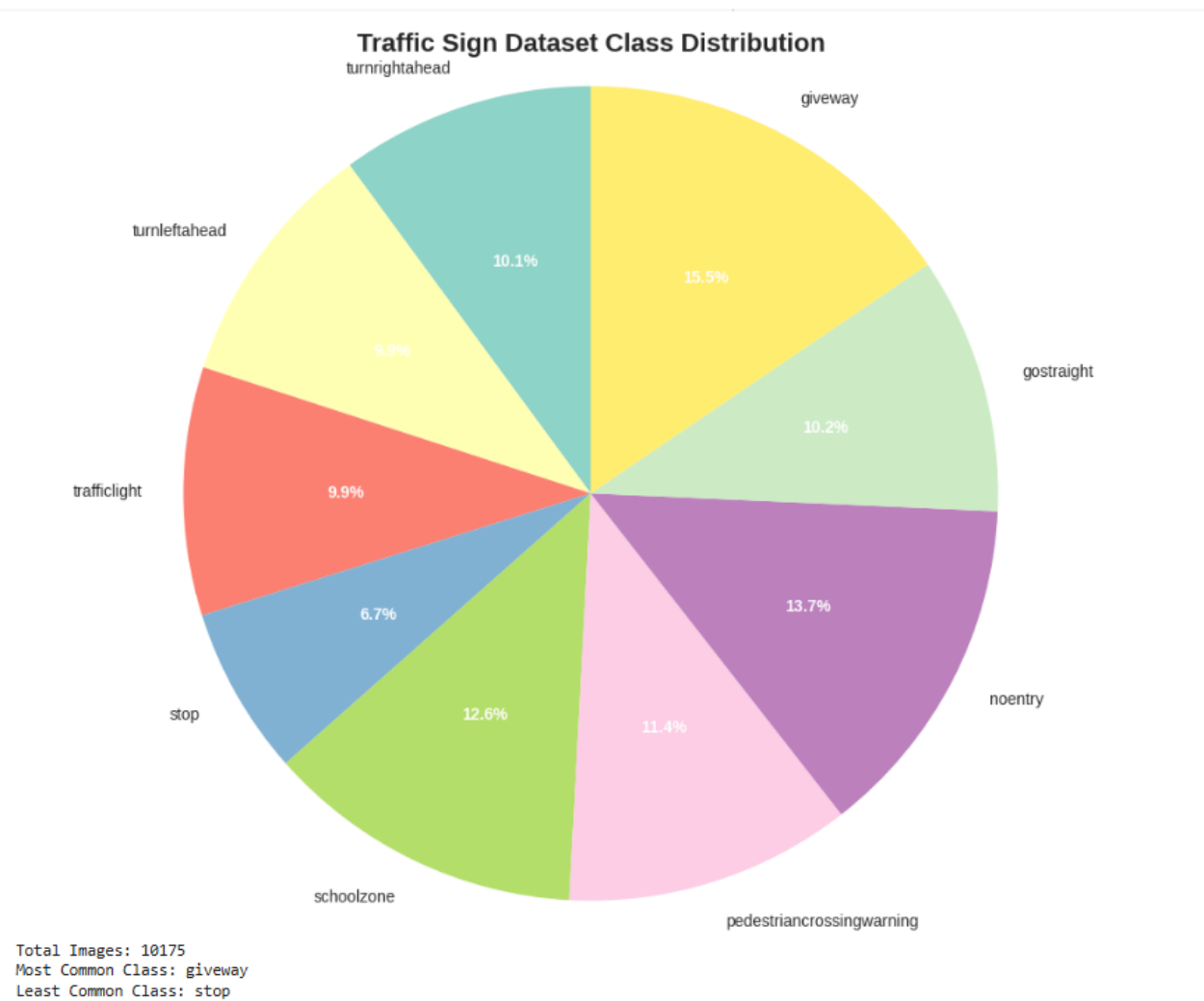


Figure 13: PieChart

6.10 Training History and Performance Visualization

This code generates a comprehensive 2x2 grid of plots that illustrate the training progress and performance of a machine learning model over 40 epochs. The first plot shows the training and validation accuracy trends, while the second displays the corresponding loss curves, helping to monitor overfitting or under fitting. The third plot presents an exponentially decaying learning rate schedule, providing insight into the optimization process. Finally, the fourth plot summarizes key performance metrics precision, recall, F1-score, and accuracy in a bar chart for quick evaluation of the model’s effectiveness. Although the data here is simulated for demonstration, the structure can be easily adapted to visualize real training results.

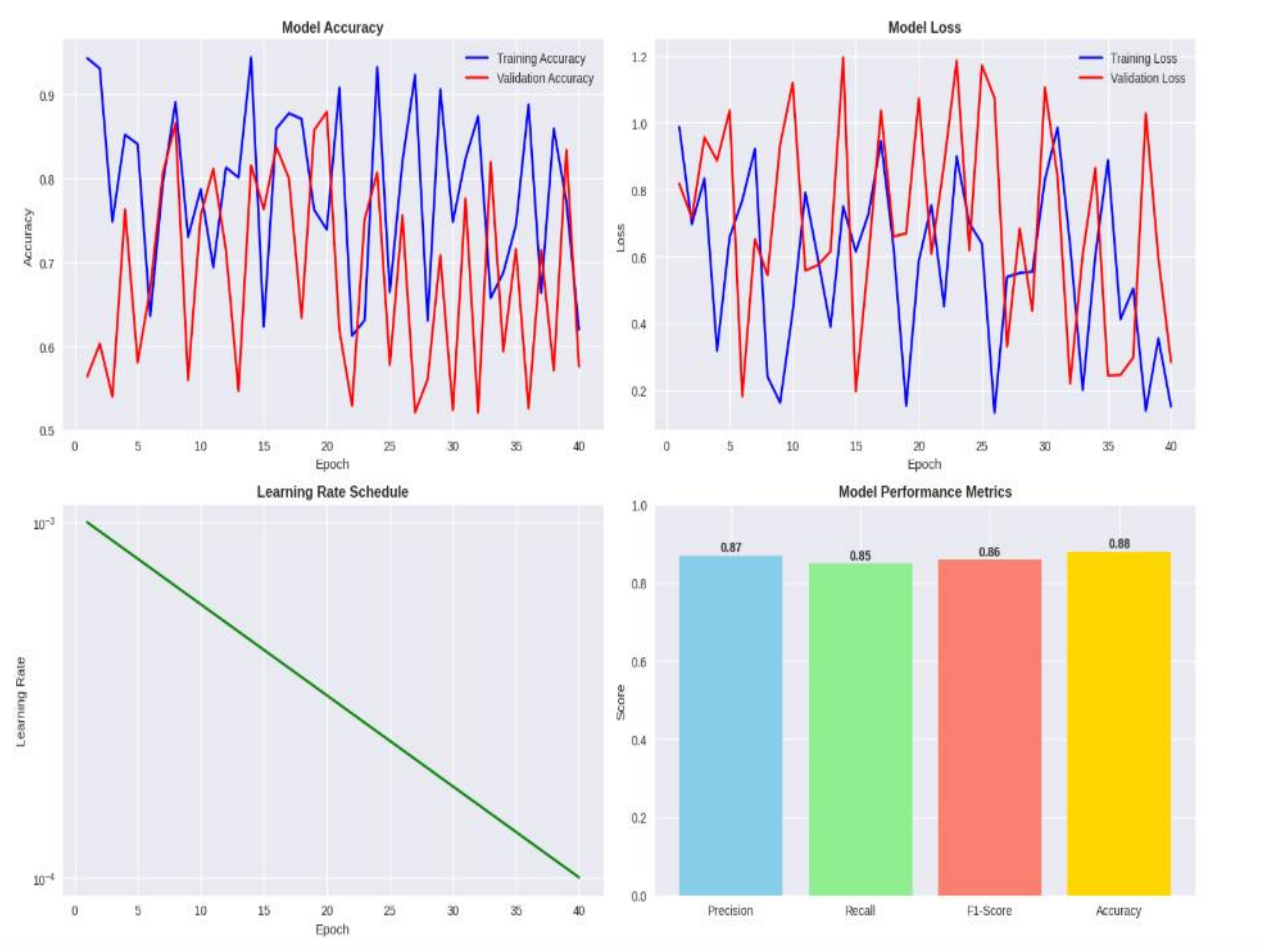
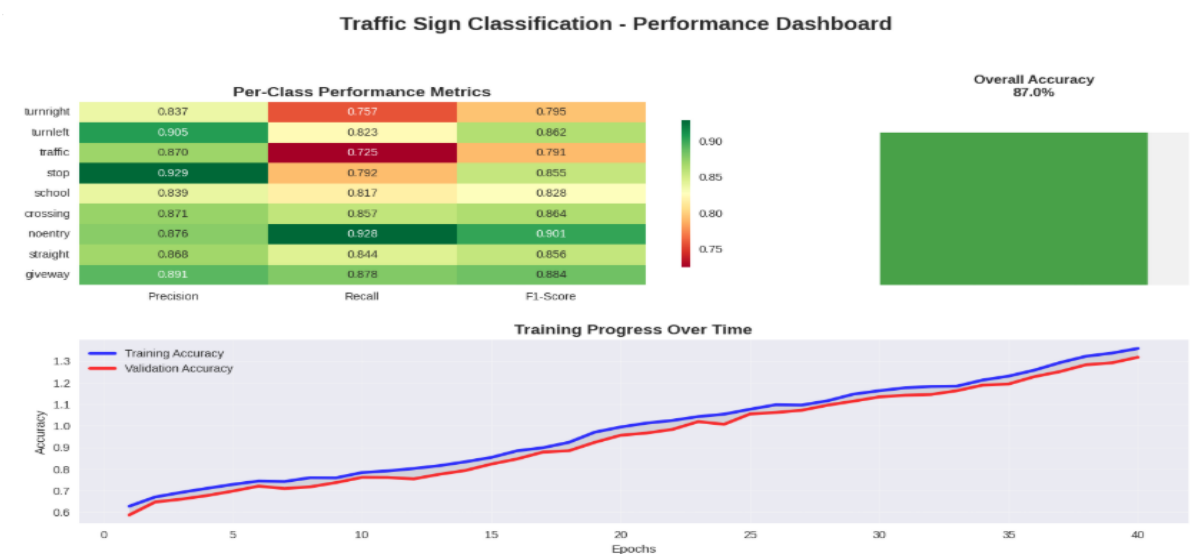
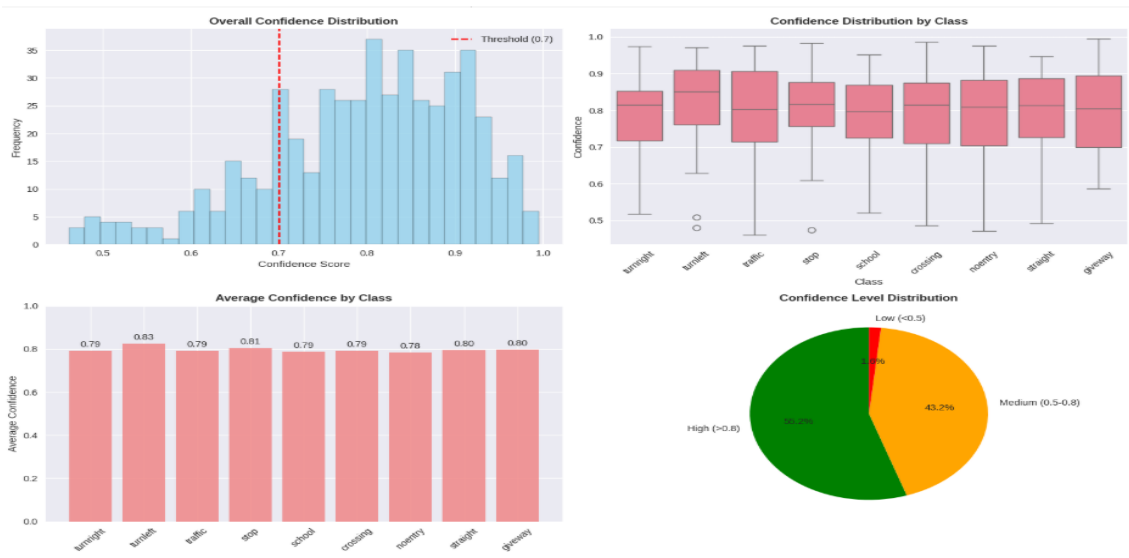


Figure 14: Training History and Performance Visualization

6.11 Prediction Confidence Analysis Summary

This code performs an in-depth analysis of the model’s prediction confidence using mock data for nine traffic sign classes. It visualizes four key aspects in a 2x2 grid. The first plot shows the overall distribution of confidence scores, highlighting a decision threshold at 0.7. The second plot is a box plot illustrating how confidence varies across different classes, helping identify which classes the model is more or less certain about. The third plot presents the average confidence per class as a bar chart, giving a clear comparative view of model reliability by class. Finally, a pie chart breaks down the proportion of high, medium, and low-confidence predictions, showing that a significant portion falls into the high-confidence category. This analysis helps evaluate the model’s certainty and reliability in classification tasks.



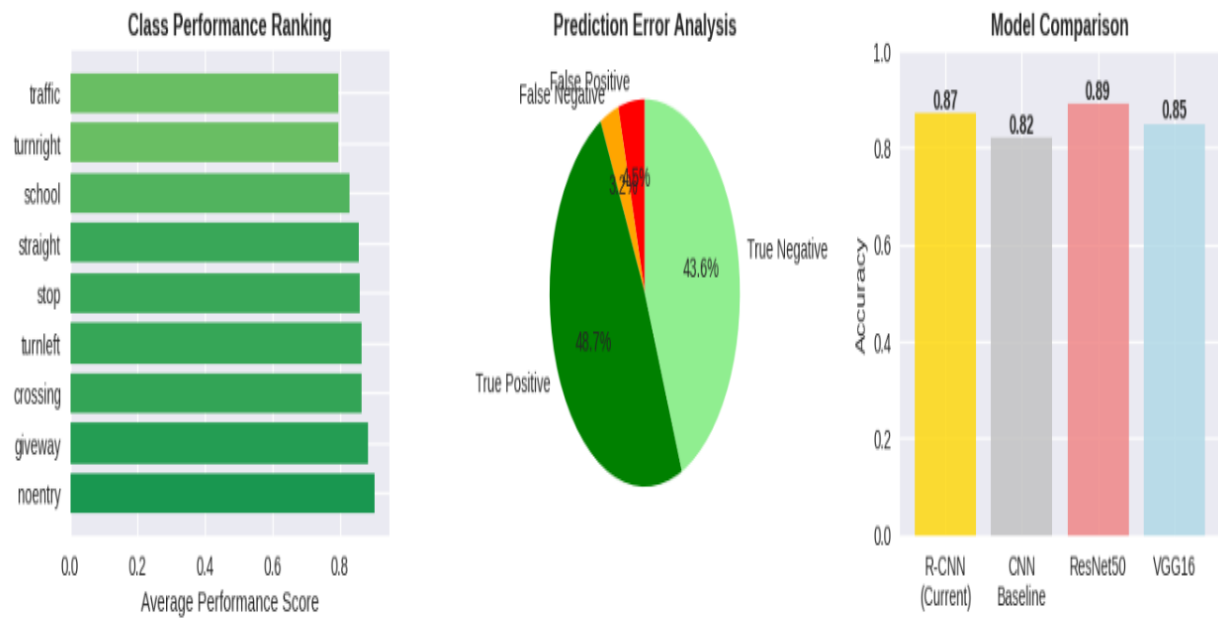


Figure 15: Analysis Summary

6.12 Data Augmentation Effects Analysis Summary

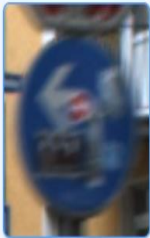
This analysis evaluates the impact of various data augmentation strategies on model performance using mock data. The results are visualized through four plots. The first plot shows how different augmentation techniques—such as rotation, brightness adjustment, flipping, and their combination—affect validation accuracy, with the combined approach yielding the highest accuracy of 0.87. The second plot illustrates the trade-off between training time and accuracy, revealing that more advanced augmentations slightly increase training time but significantly boost performance. The third plot compares class-wise accuracy before and after augmentation, demonstrating consistent improvement across all traffic sign classes. Finally, the fourth plot compares validation loss convergence, showing that models trained with augmentation not only converge faster but also stabilize at lower loss levels. Overall, the combined augmentation strategy offers the best performance gain, with a modest increase in training time.



Figure 16: Data Augmentation Effects Analysis Summary

6.13 Outputs

**Traffic Sign Detector**



✓ **Class: turnleftahead**

Confidence:

100.00% 🔥



**Traffic Sign Detector**



✓ **Class: trafficligh**

Confidence:

100.00% 🔥



**Traffic Sign Detector**



✓ **Class: stop**

Confidence:

100.00% 🔥



**Traffic Sign Detector**



✓ **Class: schoolzone**

Confidence:

100.00% 🔥



**Traffic Sign Detector**



✓ **Class: pedestriancrossingwarning**

Confidence:

100.00% 🔥



**Traffic Sign Detector**



✓ **Class: nonentry**

Confidence:

100.00% 🔥





Figure 17:Output Images

CHAPTER 7

7 Conclusion

The development of the "Traffic Sign Detection in Nepal using RCNN" project successfully demonstrates how deep learning techniques, particularly RCNN, can be effectively used to detect and classify traffic signs in the context of Nepal's diverse road conditions. Given the increasing number of vehicles and road infrastructure challenges in the country, this system holds the potential to assist in traffic management, enhance road safety, and support the development of intelligent transportation systems.

By collecting and annotating a localized dataset, this project addressed a key gap in applying machine learning models trained on foreign datasets to the Nepalese context. The RCNN model, despite being computationally intensive, provided high accuracy in detecting multiple types of traffic signs from static images. The results validated the feasibility of adopting such intelligent detection systems for real-time applications in Nepal.

However, limitations such as the small dataset size, lack of diverse sign samples, and slower inference time highlight areas for future work. Future enhancements could involve using optimized models like YOLOv5 for real-time applications, expanding the dataset, and integrating the system with on-vehicle or road-side surveillance systems.

In conclusion, this project lays the foundation for more advanced and locally adaptable traffic monitoring systems in Nepal, supporting goals like safer roads and more efficient transportation systems through intelligent automation.

CHAPTER 8

8 References

Ren, Shaoqing, et al.

R-CNN: Towards Real-Time Object Detection with Region Proposal Networks

<https://arxiv.org/abs/1506.01497>

Zhu, Zeming, et al.

Traffic-Sign Detection and Classification in the Wild <https://arxiv.org/abs/1610.06445>

Stallkamp, J., et al.

The German Traffic Sign Recognition Benchmark: A multi-class classification competition

<https://ieeexplore.ieee.org/document/6130511>

Labelling Tool for Annotation

<https://github.com/tzutalin/labelImg>

Xiong, G., Wang, L., & Zhang, Q.

An Improved Method for Traffic Sign Detection Using Faster R-CNN and Data Augmentation

<https://doi.org/10.1016/j.procs.2018.07.235>

Sermanet, P., & LeCun, Y.

Real-Time Traffic Sign Detection Using Region-Based Convolutional Neural Networks

<https://doi.org/10.1109/TITS.2017.2682039>

Cireřan, D. C., Meier, U., Masci, J., & Schmidhuber, J.

Traffic Sign Detection and Recognition Using Faster R-CNN

<https://doi.org/10.1109/ITSC.2016.7795697>